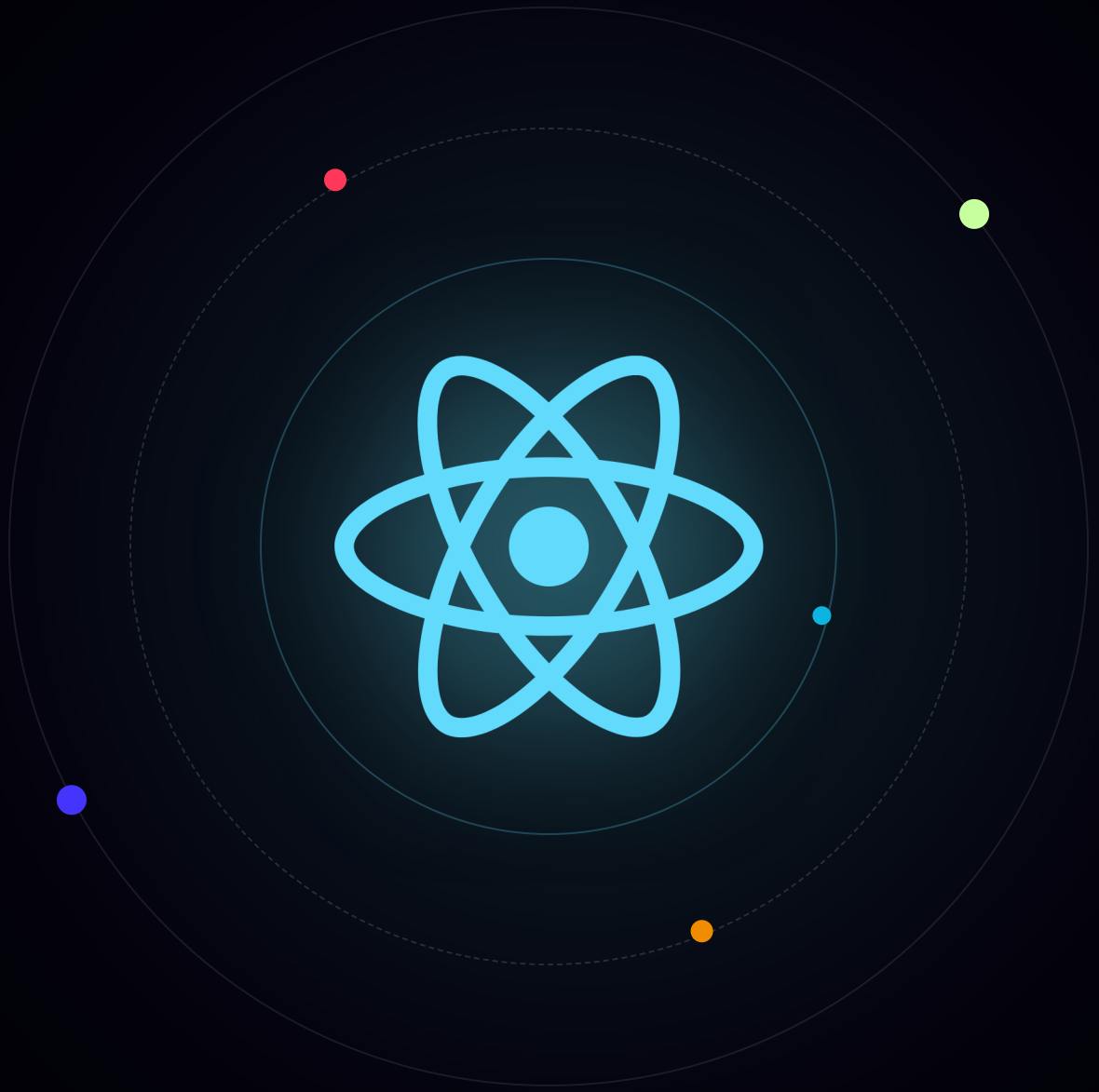
Seminarium: Implementacja Interfejsów Użytkownika

React



Bartosz Kruszewski 2026

Plan

01

Historia

skąd się wziął i dlaczego

02

Fundamenty

komponenty, stan, hooki, React 19

03

Ekosystem

biblioteki używane razem z Reactem

04

Rynek

kto używa, do czego, zalety i wady

05

Duże aplikacje

architektura i praktyki w produkcji

06

Społeczność i przyszłość

React Foundation, 19.3 i dalej

Facebook, 2011

jedna zmiana danych → wiele ręcznych poprawek w DOM



licznik w pasku nawigacji	✓ zaktualizowany
lista czatów	✓ zaktualizowana
tytuł karty przeglądarki	✓ zaktualizowany
okno czatu	✗ zapomniany

Od XHP do Reacta

Jordan Walke, inżynier Facebooka



2010

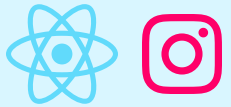
XHP

rozszerzenie PHP: komponenty i znaczniki HTML wprost w kodzie serwera

2011

FaxJS

ta sama idea w przeglądarce: przy zmianie stanu „renderuj wszystko od nowa”

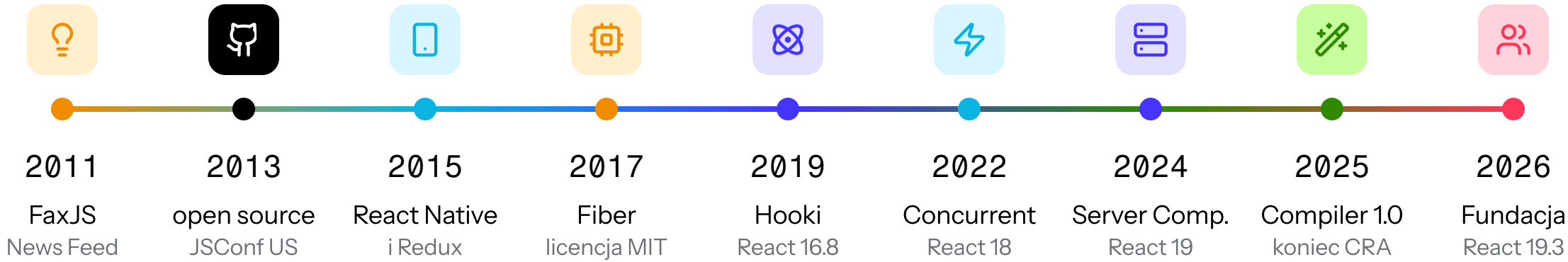


2011–2012

React

News Feed na Facebooku, potem Instagram jako pierwszy duży użytkownik

Oś czasu


2011
FaxJS
News Feed
2013
open source
JSConf US
2015
React Native
i Redux
2017
Fiber
licencja MIT
2019
Hooki
React 16.8
2022
Concurrent
React 18
2024
Server Comp.
React 19
2025
Compiler 1.0
koniec CRA
2026
Fundacja
React 19.3

2013: „Rethinking Best Practices”

JSConf US – JSX budził sprzeciw: „HTML w JavaScripcie to krok wstecz”

separacja technologii

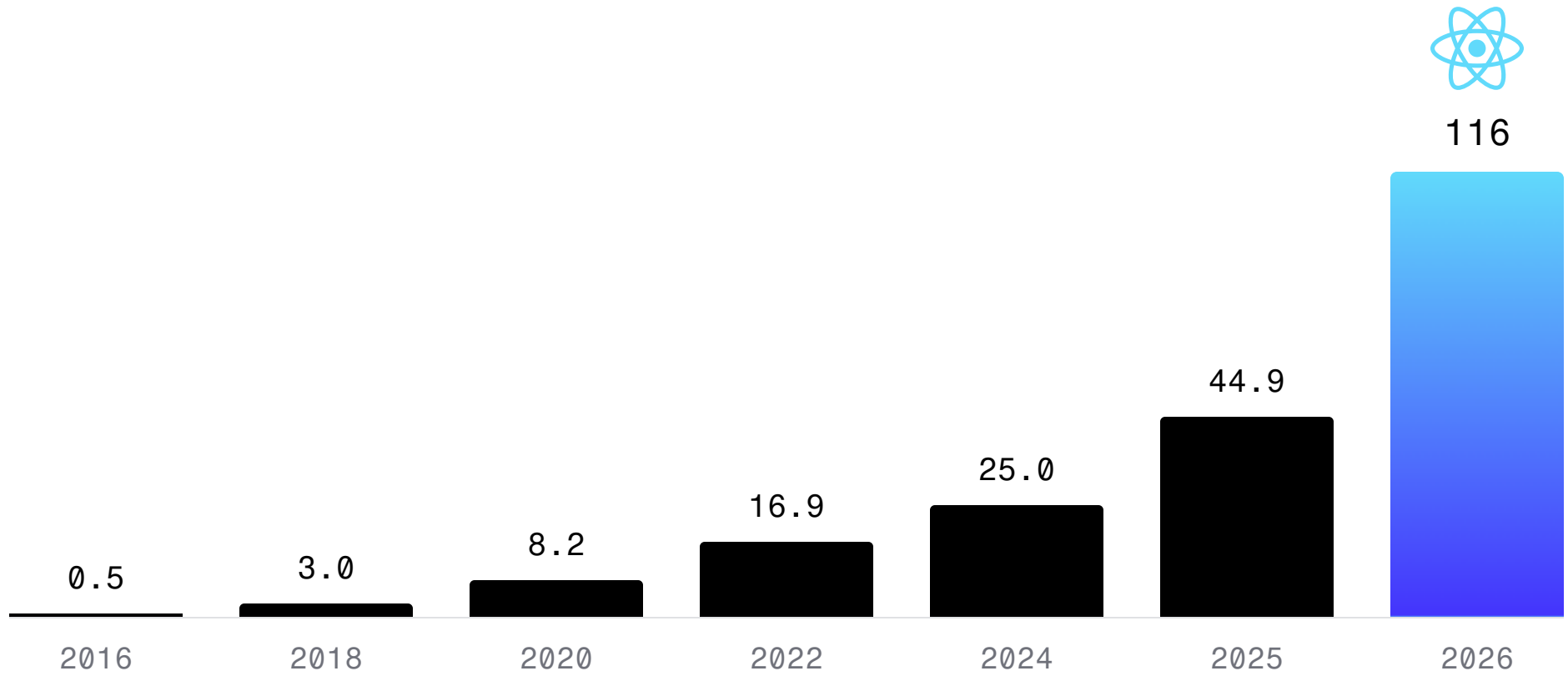


separacja odpowiedzialności – komponenty

Navbar	Feed	Chat
markup	markup	markup
logika	logika	logika
styl	styl	styl

Wzrost popularności

średnie tygodniowe pobrania pakietu react z npm, w milionach



npm registry API · wrzesień danego roku, 2026: tydzień do 18.10 · pobrania obejmują CI i boty

Główna idea

$$UI = f(state)$$

opisujesz **jak ma wyglądać** interfejs, React ustala **co zmienić**

stan → komponent → DOM

Imperatywnie vs deklaratywnie

ten sam przycisk „Lubię to” na dwa sposoby

jQuery – jak to zrobić

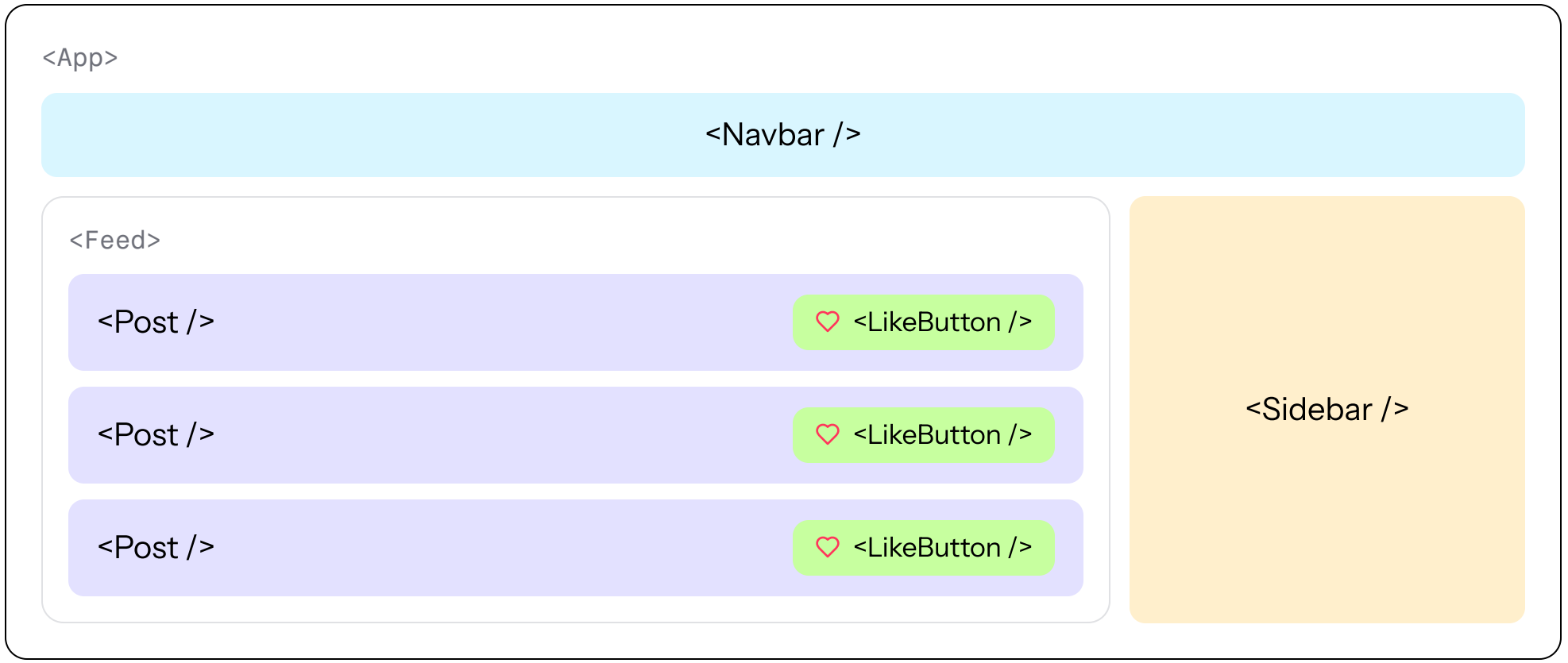
```
$('#like').on('click', () => {
  liked = !liked;
  $('#like').toggleClass('active');
  $('#like .label')
    .text(liked ? 'Lubię' : 'Polub');
  $('#count').text(liked ? n + 1 : n);
});
```

React – co ma być widać

```
function Like({ n }) {
  const [liked, setLiked] = useState(false);
  return (
    <button onClick={() => setLiked(!liked)}>
      {liked ? 'Lubię' : 'Polub'}
      {liked ? n + 1 : n}
    </button>
  );
}
```

Komponenty

UI to drzewo małych, wielokrotnie używanych funkcji



JSX

składnia podobna do HTML, kompilowana do zwykłych wywołań funkcji

</> to piszesz

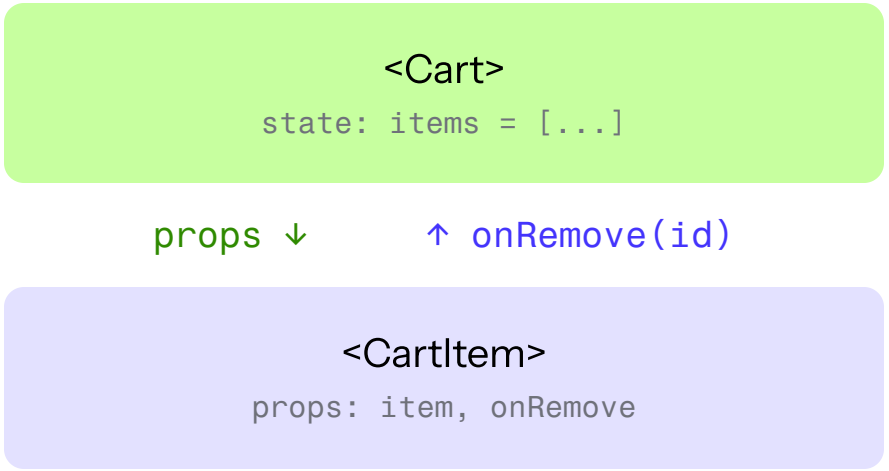
```
<Post author="Ala" likes={3}>
  <h2>Cześć!</h2>
</Post>
```


to dostaje przeglądarka

```
jsx(Post, {
  author: "Ala",
  likes: 3,
  children: jsx("h2", {
    children: "Cześć!"
  })
})
```

Props i stan

dane płyną w dół, zdarzenia w górę



props

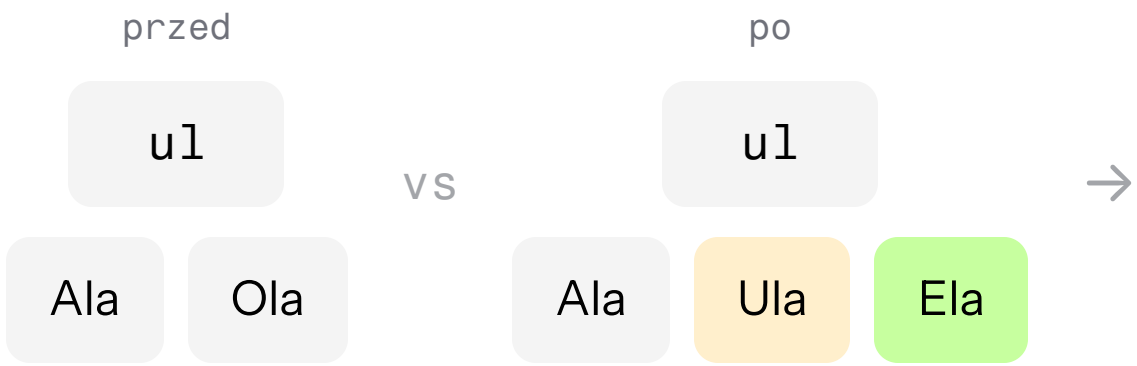
wejście komponentu od rodzica, tylko do odczytu

state

pamięć komponentu; zmiana przez `setState` powoduje ponowny render

Virtual DOM i rekonsyliacja

React porównuje stare i nowe drzewo, w DOM aplikuje tylko różnicę



operacje na DOM

- `li[1].text = "Ula"`
- `ul.append(li "Ela")`

2 operacje zamiast przebudowy całej listy

Fiber (React 16, 2017)

renderowanie można przerwać i wznowić

stary silnik – jeden nieprzerywalny blok



Fiber – praca w kawałkach, pilne zdarzenia wchodzą w środek



czas →

Hooki (16.8, 2019)

logika ze stanem w zwykłych funkcjach zamiast klas

✕ przed: komponent klasowy

```
class Counter extends Component {
  state = { n: 0 };
  componentDidMount() {
    document.title = this.state.n;
  }
  componentDidUpdate() {
    document.title = this.state.n;
  }
  render() { /* ... */ }
}
```

✓ po: hooki

```
function Counter() {
  const [n, setN] = useState(0);

  useEffect(() => {
    document.title = n;
  }, [n]);

  return <button onClick={() =>
    setN(n + 1)}>{n}</button>;
}
```

Concurrent rendering (React 18)

nie każda aktualizacja jest równie pilna

```
const [pending, startTransition] =
  useTransition();

function onChange(e) {
  setQuery(e.target.value);    // pilne
  startTransition(() => {
    setFilter(e.target.value); // w tle
  });
}
```

wpisywany tekst

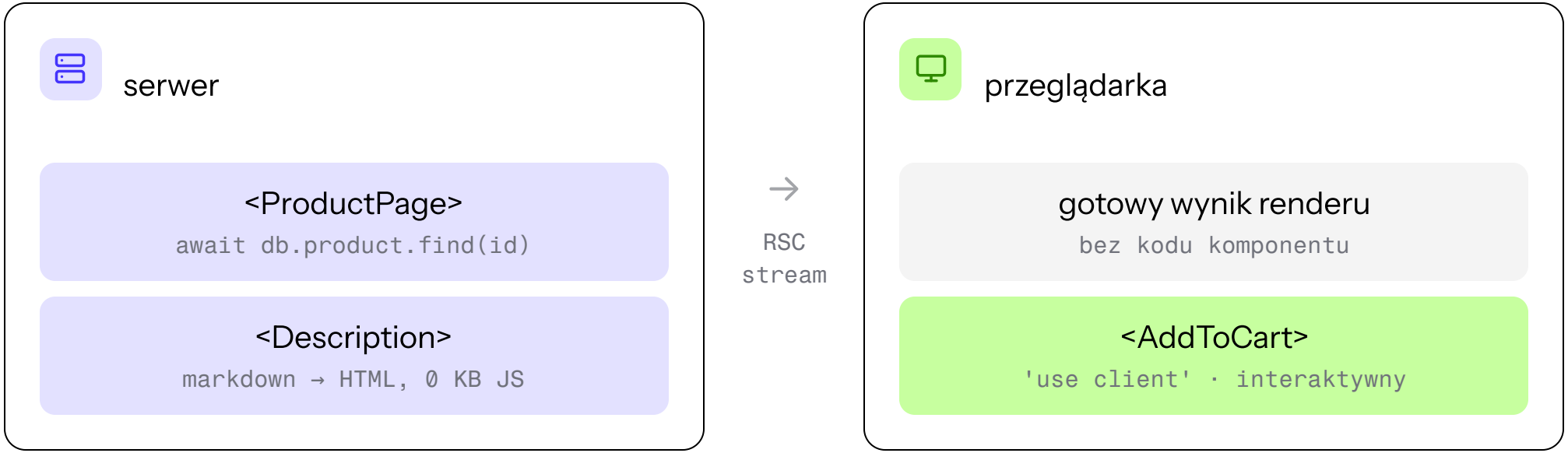
pilne – pole reaguje natychmiast

filtrowanie 10 000 wierszy

transition – liczone w tle, przerywane przy kolejnym znaku

Server Components (React 19)

część komponentów działa tylko na serwerze i nie trafia do bundla



Actions i formularze (React 19)

asynchroniczne akcje jako wbudowany element API

```
const [state, action, pending] =
  useActionState(async (prev, form) => {
    return await subscribe(form.get('email'));
  }, null);

<form action={action}>
  <input name="email" />
  <button disabled={pending}>Zapisz</button>
</form>
```

useActionState

wynik, błąd i stan „pending”

useOptimistic

UI zmienia się, zanim serwer odpowie

use()

odczyt Promise lub kontekstu w renderze

React Compiler 1.0 (2025)

automatyczna memoizacja w czasie kompilacji

przed: ręczna optymalizacja

```
const sorted = useMemo(
  () => items.toSorted(byPrice),
  [items]
);
const onSelect = useCallback(
  (id) => setSelected(id), []
);
export default memo(ProductList);
```

z kompilatorem

```
const sorted = items.toSorted(byPrice);
const onSelect = (id) => setSelected(id);
export default ProductList;
```

do 12% szybsze ładowanie

Meta Quest Store; część interakcji ponad 2,5× szybsza

React to biblioteka, nie framework

resztę dobiera się z ekosystemu



framework

Next.js · React Router

routing, SSR, RSC, build



dane z serwera

TanStack Query · SWR

cache, ponawianie, odświeżanie



stan klienta

Zustand · Redux Toolkit

globalny stan aplikacji



formularze

React Hook Form · Zod

walidacja i typy z jednego schematu



UI i style

Tailwind · shadcn/ui

Radix, MUI, CSS Modules



testy

Vitest · Testing Library

Playwright, Storybook

Frameworki i narzędzia startowe

Create React App wygaszony w lutym 2025 – oficjalnie zalecany jest framework



Vercel

Next.js

App Router, Server Components, SSR/SSG



Shopify

React Router 7

dawniej Remix; od SPA po pełny framework



TanStack

TanStack Start

w pełni typowany routing



Expo

Expo

framework dla React Native

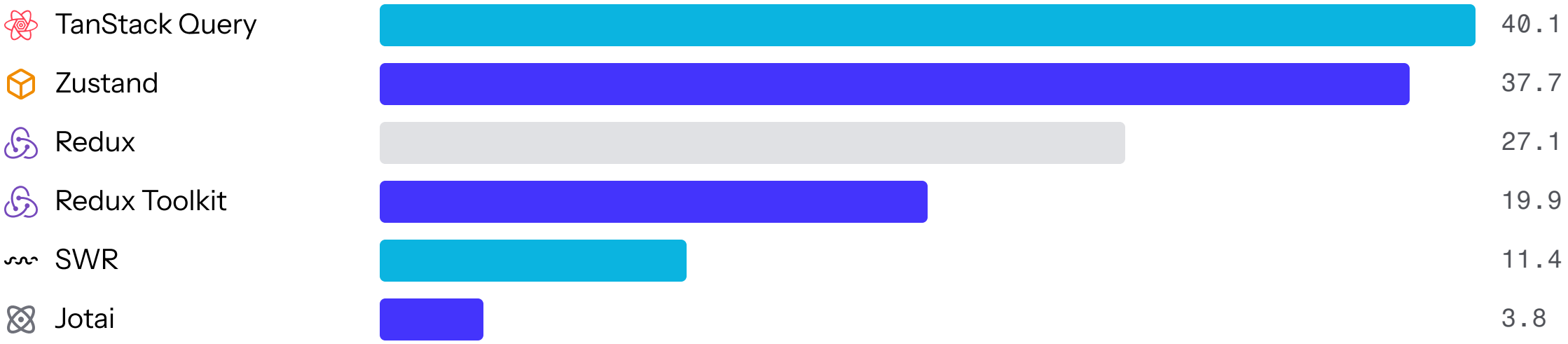


Vite `119 mln pobrań / tydz.`

czysty SPA bez serwera: Vite + React + router

Zarządzanie stanem

tygodniowe pobrania z npm, w milionach



Stylowanie i komponenty UI

Tailwind CSS – 85 mln pobrań tygodniowo

```
<button className="
  rounded-lg bg-black
  px-4 py-2 text-white
  hover:bg-zinc-800">
  Zapisz
</button>
```

klasy narzędziowe zamiast osobnych plików CSS



shadcn/ui

kod komponentów kopiowany do repozytorium



Radix UI · React Aria

„headless” – dostępność bez wyglądu



MUI · Ant Design

gotowe, kompletne zestawy komponentów

Testowanie

„Im bardziej testy przypominają sposób użycia aplikacji, tym więcej dają pewności”



E2E · Playwright

prawdziwa przeglądarka



komponenty · React Testing Library

render i interakcje jak użytkownik





jednostkowe · Vitest / Jest

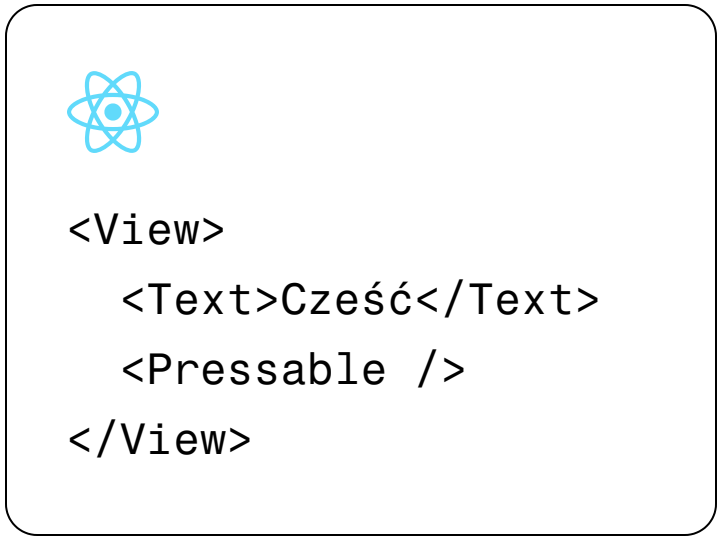
hooki, logika, funkcje pomocnicze



Storybook – katalog komponentów i testy wizualne

React Native

ten sam model komponentów, natywne widoki zamiast DOM



iOS

Android

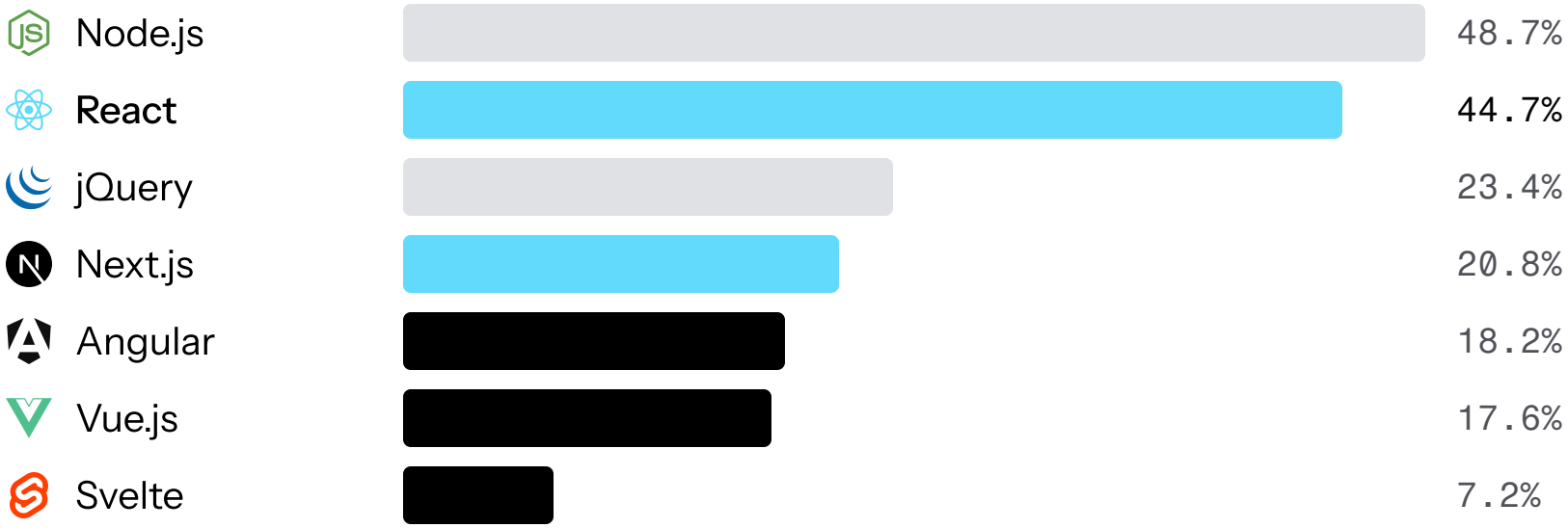
Windows, macOS

Web, TV, Quest

używają m.in.

Najpopularniejsze technologie webowe

Stack Overflow Developer Survey 2025 · % respondentów



11×

więcej pobrań z npm niż Vue

116 mln vs 10 mln / tydz.

Kto używa Reacta



Meta

Facebook, Instagram, Threads



Microsoft

Outlook, Teams, Office



Netflix

web i interfejsy na TV



Airbnb

web i design system



Discord

desktop, web, mobile



Atlassian

Jira, Confluence



Shopify

aplikacje mobilne



Notion · Vercel

i tysiące innych

Do czego się nadaje

✓ dobry wybór

- aplikacje SPA i SaaS
- panele i dashboardy
- e-commerce z SSR (Next.js)
- mobile i desktop (RN, Electron)
- design systemy wielu zespołów

✕ są lepsze opcje

- blog, dokumentacja → Astro
- proste strony z serwera → htmx
- mały widget → Preact, Svelte
- gry i grafika → Canvas, WebGL

Zalety

Prosty model
UI = f(state), komponenty to funkcje

Ekosystem
gotowa biblioteka na niemal każdy problem

Rynek pracy
najwięcej ofert i developerów

Wiele platform
web, mobile, desktop, VR

Stabilność
łagodne migracje, testowany na produktach Meta

Narzędzia i AI
DevTools, TypeScript; modele AI znają go najlepiej

Wady



Zmęczenie decyzjami

routing, dane, style, stan – każdy projekt wygląda inaczej



Pułapki wydajności

zbędne re-rendery, nadużywany useEffect



Rosnąca złożoność

Server Components, 'use client', cache – trudniejsze niż klasyczny SPA



Rozmiar i szybkość

virtual DOM cięższy niż sygnały (Solid, Svelte 5)

Cena złożoności: React2Shell

CVE-2025-55182 · zdalne wykonanie kodu w Server Components

10.0

CVSS – maksymalna ocena

3.12.2025	publikacja i poprawki w 19.0.1, 19.1.2, 19.2.1
5.12.2025	CISA: aktywnie wykorzystywana w atakach
11.12.2025	dwie kolejne podatności: DoS i wyciek kodu

przyczyna: niebezpieczna deserializacja w protokole Flight – wystarczyło, że aplikacja wspierała RSC

Struktura według funkcji

feature-based zamiast podziału według typu pliku

✕ według typu

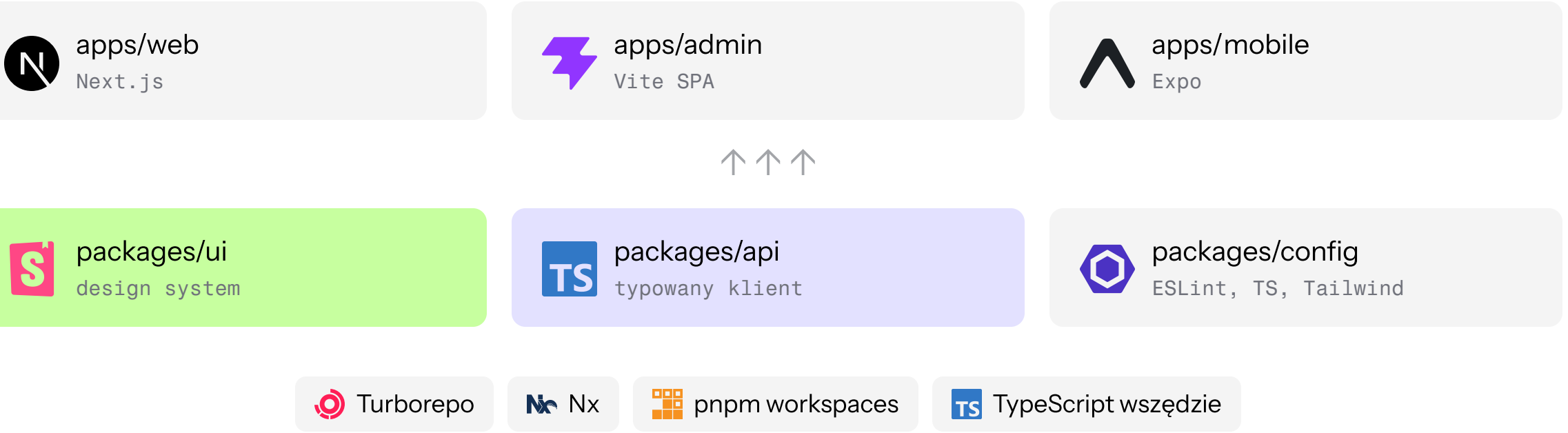
```
src/
  components/
    (200 plików)
  hooks/
  services/
  utils/
```

✓ według funkcji

```
src/
  features/
    checkout/
      components/ api.ts
      index.ts   ← publiczne API
    orders/
  shared/ui/   ← design system
  app/         ← routing, providery
```

Monorepo i design system

kilka aplikacji, wspólne komponenty i konfiguracja



Strategie renderowania

gdzie i kiedy powstaje HTML

	gdzie	kiedy	typowe użycie
CSR	przeglądarka	przy każdym wejściu	panel za logowaniem
SSR	serwer	przy każdym żądaniu	dane użytkownika + SEO
SSG	serwer	w czasie builda	dokumentacja, landing
ISR	serwer	build + odświeżanie	katalog produktów
RSC	serwer, per komponent	przy żądaniu, stream	treść + interakcje

Społeczność w liczbach

251k
gwiazdek na GitHubie

116M
pobrań z npm tygodniowo

44.7%
developerów używa Reacta

8
firm w React Foundation

React Conf · React Summit · This Week In React · react.dev · meetupy w Polsce

React Foundation

od 24.02.2026 React należy do Linux Foundation, nie do Meta

Meta

przekazuje React,
React Native i JSX

→

React Foundation

Amazon	Callstack	Expo	Huawei
Meta	Microsoft	Software Mansion	Vercel

firmy z Polski

React 19.3 (wrzesień 2026)

View Transitions

animacje wejścia, wyjścia i przejść

Fragment Refs

ref na grupę elementów: fokus, zdarzenia

browser()

komponent renderowany tylko w przeglądarce

Trusted Types

lepsza ochrona przed XSS

Niezależne transitions

wolna aktualizacja nie blokuje innych

Context w RSC

bez wrapperów z 'use client'

Kierunki rozwoju

ręczna optymalizacja	kompilator robi to za ciebie
wszystko w przeglądarce	serwer i klient jako jedno drzewo
biblioteki do animacji	wbudowane View Transitions, także w RN
projekt Meta	fundacja i otwarte zarządzanie

Podsumowanie

01

Jedna idea

UI = f(state) – niezmienna od 2013 roku, zmieniają się narzędzia

02

Standard rynkowy

najpopularniejszy framework frontendowy z największym ekosystemem

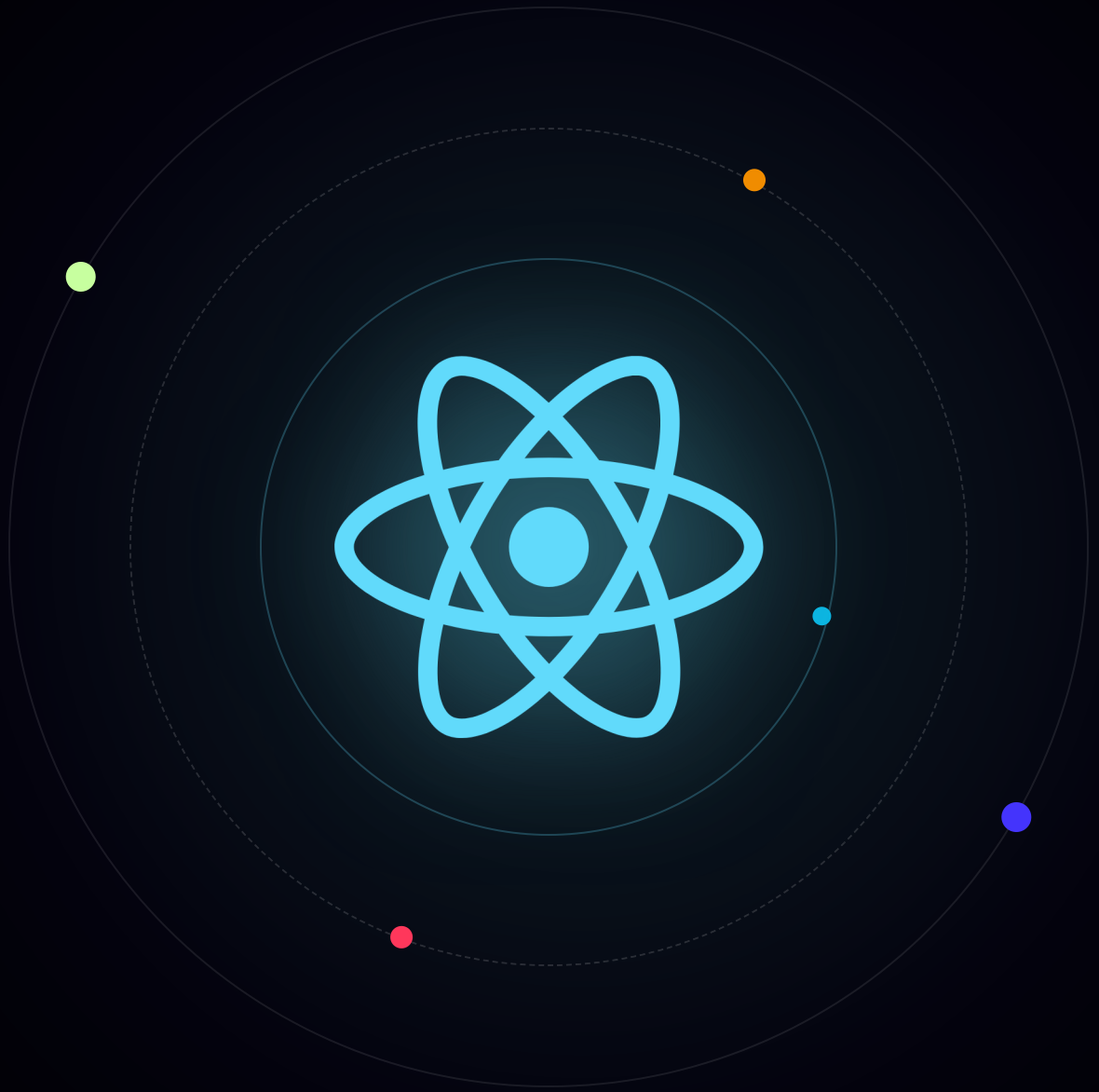
03

Dojrzewa

kompilator, serwer, fundacja – mniej ręcznej pracy, więcej złożoności pod spodem

Dziękuję

Pytania?



Źródła

- React Blog
  react.dev/blog · 19.x, Compiler, CRA
- React Foundation
  linuxfoundation.org/press
- React2Shell, CVE-2025-55182
  react.dev/blog/2025/12/03
- Developer Survey 2025
  survey.stackoverflow.co/2025
- Statystyki pobrań npm
  api.npmjs.org
- Repozytorium Reacta
  github.com/react/react
- „Rethinking Best Practices”
  Pete Hunt · JSConf US 2013
- Logotypy i ikony
  simpleicons.org · lucide.dev